



Design, implementation and evaluation of Reproducible Self-Healing Micro-Batch ETL Pipeline for Smart-City Traffic Data

Author: Birhane Gebrewahid

Program: Master of Science in Data Engineering

Institution: Constructor University

Supervisors: Dr. Hamza Salem & Dr. Fangning Hu

Master's Thesis Defense

August 27, 2026

Problem & motivation

Reliable traffic analytics requires more than collecting data.

Heterogeneous data

External traffic data varies in structure, format, and update behavior. Raw availability does not guarantee analytical usability.

Operational failures

API, processing, and infrastructure failures can interrupt recurring workflows and reduce data availability.

Geographic variation

City-specific parameters can create duplicated implementations unless configuration is separated from common processing logic.

Research gap

An integrated architecture is needed for reproducible deployment, workflow recovery, and reusable multi-city processing.

Objectives & Research questions

The thesis evaluates one system against three design objectives.

Main objectives

Reproducibility

Containerization + infrastructure automation.

Resilience

Orchestration + monitoring + recovery.

Reusability

Configuration + standardized schemas.

Research Questions

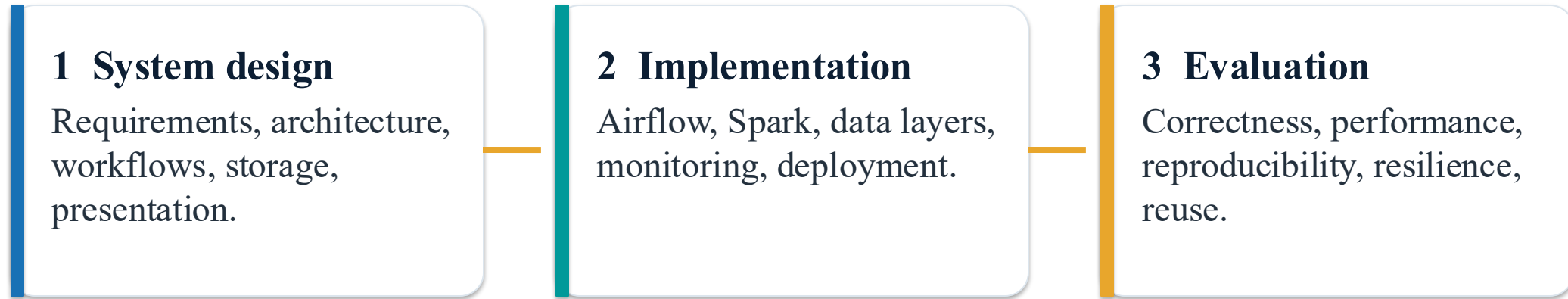
RQ1 How can an ETL pipeline for smart-city traffic data achieve reproducible deployment and execution across different environments?

RQ2 How can an ETL pipeline detect and recover from failures during recurring traffic data ingestion and processing with minimal manual intervention?

RQ3 How can an ETL pipeline support ingestion and processing of traffic data from multiple cities using a common and reusable data model?

Research method & scope

Design → implementation → evaluation.

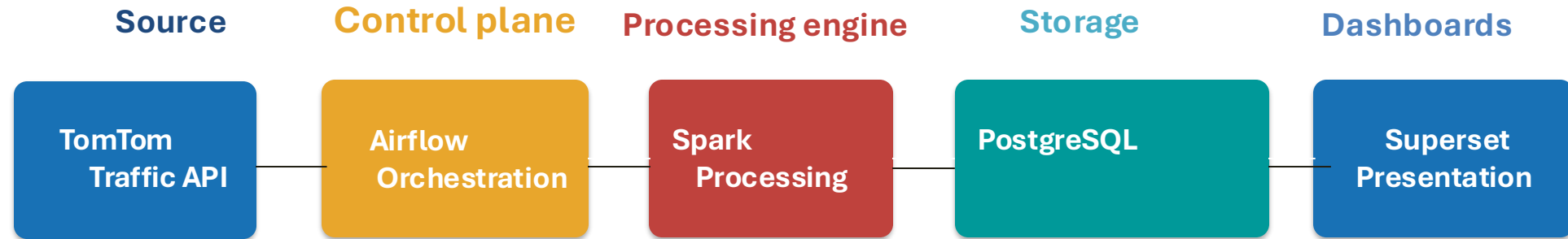


Scope boundaries

- Traffic flow and incident ETL from the TomTom Traffic API.
- Recurring micro-batch processing, not fully streaming.
- No traffic prediction, route optimization, or machine-learning forecasting.

End-to-end system architecture

Six logical components connect collection to analytical consumption.



Deployment boundary

Terraform provisions AWS EC2 → Docker Compose runs Airflow, Spark, PostgreSQL, and Superset

TomTom flow & incident ETL

Two independent data paths share the same platform principles.

Stream	TomTom service	Raw representation	Downstream use
Traffic flow	FlowSegment API	JSONL batch files	Traffic flow analysis + hourly execution
Incidents	IncidentDetails API	JSONL batch files	Incident analysis + critical-event alerts +15 min execution

Representative source fields

- Flow: city, batch timestamp, coordinates, current speed, free-flow speed, travel time, confidence, road closure.
- Incidents: city, incident type/category, coordinates, delay, road numbers, start/end timestamps.

Bronze - Silver - Gold architecture

Spark progressively refines raw data into analytical outputs.

Layer	Purpose	Implemented processing
Bronze	Raw-data preservation	Schema parsing; invalid geographic-record filtering; ingestion metadata; partitioned Parquet
Silver	Clean and standardize	Null handling, timestamps, deduplication, feature engineering
Gold	Analytical consumption	Hourly aggregation, point-level outputs, KPI outputs, latest incidents

Airflow orchestration vs Spark processing

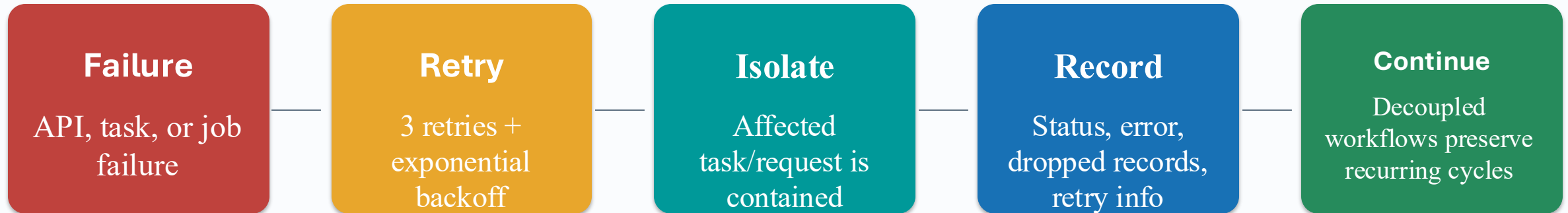
The architecture makes the control/data-plane boundary explicit

Airflow - orchestration	Spark - processing
Schedules recurring workflows	Reads and transforms data
Defines task dependencies	Cleans and feature-engineers data
Triggers and monitors Spark jobs	Writes Bronze, Silver, and Gold outputs
Retries, backoff, failure handling	Produces analytical datasets
Records task states and logs	Processes Parquet partitions

Airflow does not perform the transformations; it coordinates the jobs that Spark executes.

Self-healing & failure recovery

Recovery is implemented at task/workflow level.

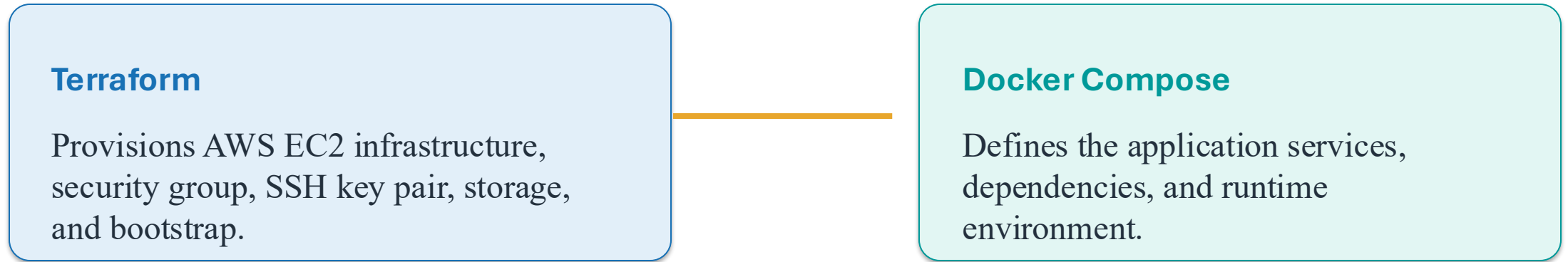


Total DAG runs	Successful	Failed	Recovered/retried
270	257	10	3

Controlled task execution and monitoring support recovery without requiring manual restart.

Reproducible deployment

Terraform and Docker Compose address different reproducibility layers.



Qualification: AWS evaluated deployment reproducibility; complete ETL execution was evaluated locally because the selected cloud instance had limited resources.

Approach	Estimated effort
Manual deployment	61 minutes
Terraform-based	10.5 minutes
Estimated reduction	82.8%

Multi-city configuration & reuse

City-specific geography is separated from shared processing logic.

Berlin

bbox + grid_steps = 5 →
25 points

Bremen

bbox + grid_steps = 4 →
16 points

Frankfurt

bbox + grid_steps = 5 →
25 points

Same Airflow workflows + same Spark jobs + standardized schemas

City	Ingested	Processed	Dropped
Berlin	6.86k	2.33k	728
Bremen	6.86k	1.49k	726
Frankfurt	6.86k	2.33k	726

Testing & validation

Tests verify configuration behavior and city-grid integration.

Test area	Result
Expected cities in CITY_CONFIG	PASS
Valid configuration accepted	PASS
Invalid configuration rejected	PASS
Benchmark smoke test	PASS
CITY_CONFIG + build_grid() integration	PASS

Automated test suite

5 tests

All passed

Run time

0.16 s

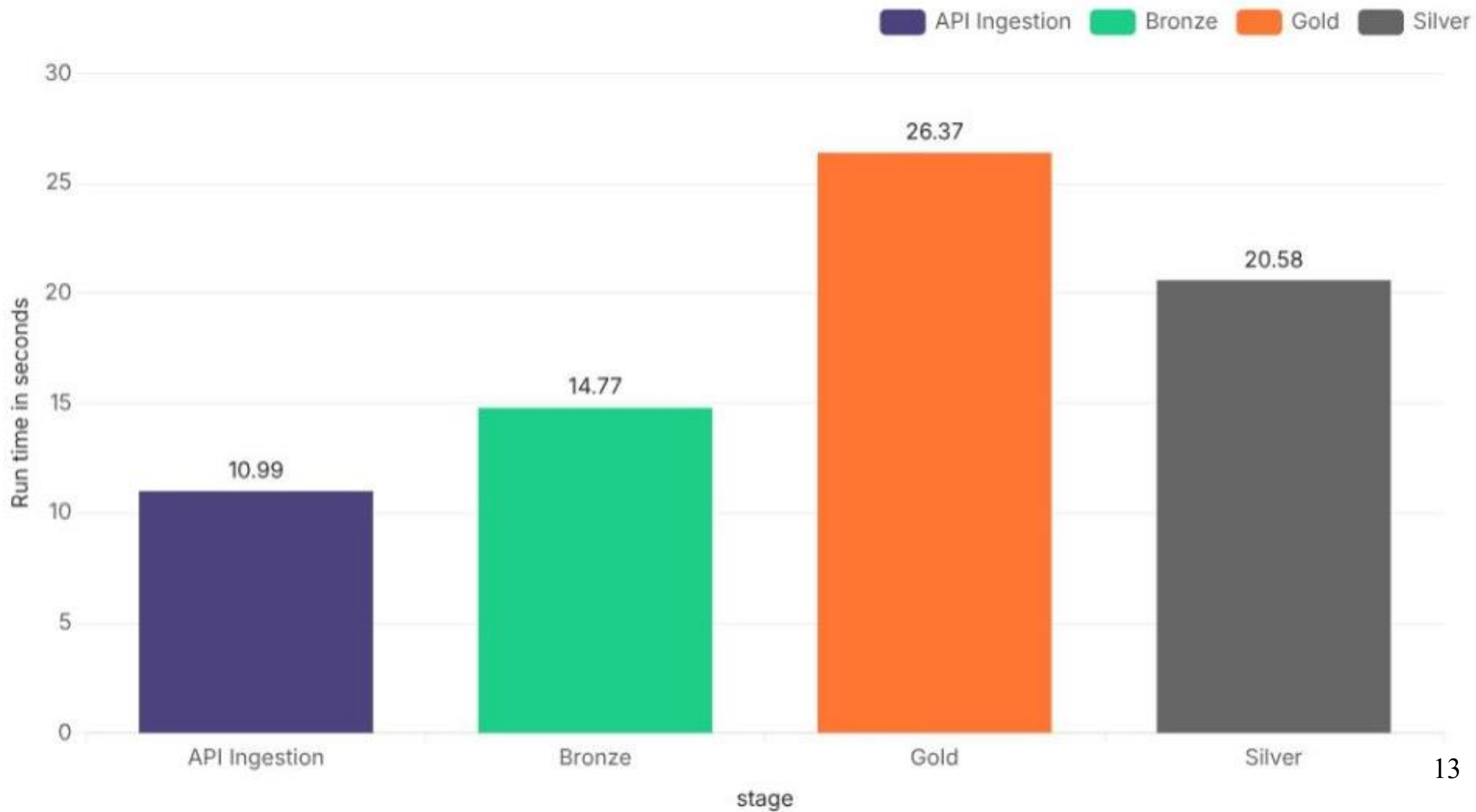
Integration

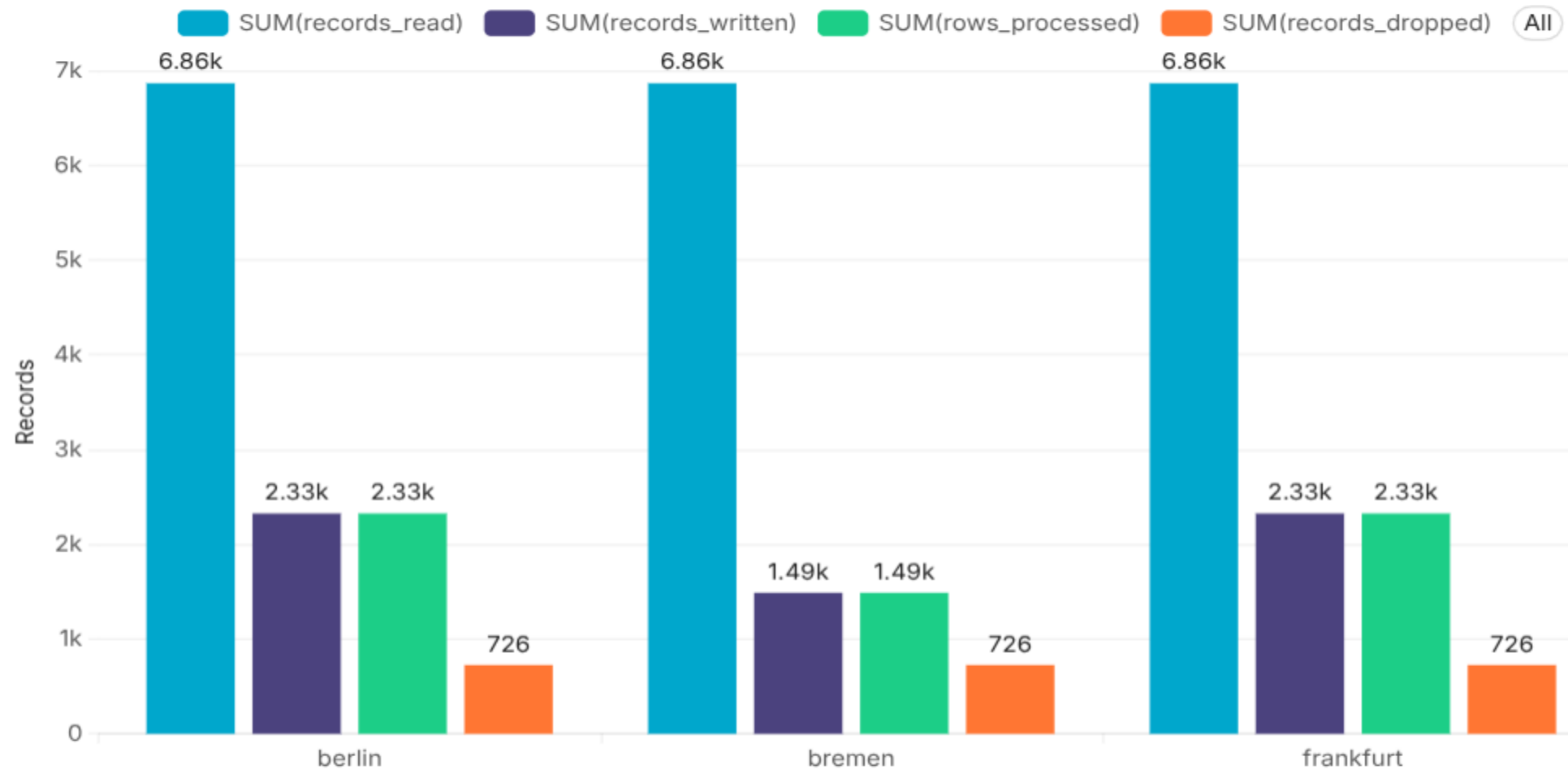
3 cities validated

Performance & operational results

The pipeline reports component-level benchmarks and execution evidence.

City	Grid points	Execution time
Berlin	25	0.000022 s
Bremen	16	0.000012 s
Frankfurt	25	0.000015 s
Total	66	0.000080 s





Findings for RQ1–RQ3

Each research question is answered by a mechanism and evaluation evidence.

RQ	Mechanism	Evidence / finding
RQ1	Docker Compose + Terraform	61 → 10.5 min estimated deployment effort; 82.8% reduction
RQ2	Airflow retries, backoff, isolation, monitoring	270 DAG runs; 3 recovered/retried tasks
RQ3	Configuration-driven city processing + common schemas	Same architecture processed Berlin, Bremen, Frankfurt

The findings support reproducibility, resilience, and reusability within the evaluated scope.

Contributions, limitations & future work

The thesis delivers an engineering foundation, not a complete traffic-intelligence product.

Contributions

Reproducible traffic data-engineering architecture.

Self-healing micro-batch workflow.

Configuration-driven multi-city processing.

Evaluation framework across five criteria.

Limitations

Three cities only.

Micro-batch rather than fully streaming.

Limited cloud resources constrained workload evaluation.

No prediction or machine-learning analytics.

Future work

Continuous event-driven streaming.

Prediction, forecasting using machine learning algorithms

Cloud performance evaluation with larger resources.

Conclusion

Reproducible | resilient | reusable foundation for smart-city traffic pipeline

1. TomTom flow and incident data - transformed into analytical outputs.
2. Airflow coordinates execution and recovery, Spark performs transformation.
3. Docker and Terraform address application and infrastructure level reproducibility.
4. Configuration-driven processing demonstrates reuse across three cities.

Thank you | Questions?

Demo